AOS1 A2026
TP 2 — Bayesian linear regression

`numpy=1.26.4; seaborn=0.13.2; matplotlib=3.10.1; pandas=2.2.0`

1 Introduction

This practical session aims at applying Bayesian linear regression. You will need several libraries for this purpose.

```
import numpy as np
import scipy as sp
import scipy.stats as spst
import matplotlib.pyplot as plt
```

2 Programming Bayesian linear regression

In this section, we aim at implementing a class for Bayesian linear regression which satisfies the `sklearn` structure.

1 Program a class which contains an initialization method (given below), a function for fitting the model, and a function for making predictions, possibly including the estimation of the variances. You may use the `np.linalg.inv` function for the fitting function: optimizing the computation (using, e.g., Cholesky decomposition) is not mandatory—the purpose is mainly for you to have a good understanding of the model (yet, you're welcome to optimize it as much as you want).

```
class BayesianLR:
    def __init__(self, alpha=1.0):
        self.alpha = alpha
        self.coef_ = None
        self.intercept_ = None
        self.sigma_sq = None  # noise variance
        self.beta_cov = None  # covariance matrix of coefficients

    def fit(self, X, y):
        ...

    def predict(self, X, return_std=False):
        ...
```

2 Generate a synthetic regression dataset. You can use the `make_regression` and `train_test_split` functions from `scikit-learn`. You can scale the data to avoid numerical issues when fitting the model (function).

3 Fit the model to the data generated. Display the outputs together with the associated credibility intervals; for this purpose, you can use the code below (where `Xte_` stands for the scaled input data).

```
sort_idx = np.argsort(Xte_.ravel())
Xte_sorted = Xte_[sort_idx]
y_ave_sorted = y_ave[sort_idx]
y_std_sorted = y_std[sort_idx]

plt.figure(figsize=(10, 6))
plt.scatter(Xte_, yte, color='black', alpha=0.6, label='Observed data')
plt.plot(Xte_sorted, y_ave_sorted, color='blue', lw=2, label='Predicted
↪  outputs')
plt.fill_between(Xte_sorted.ravel(),
                 y_ave_sorted - 1.96 * y_std_sorted,
                 y_ave_sorted + 1.96 * y_std_sorted,
                 color='blue', alpha=0.2, label='95% credibility interval')

plt.title("Bayesian Linear Regression")
plt.xlabel("Standardized inputs")
plt.ylabel("outputs")
plt.legend()
plt.show()
```

4 Repeat the procedure above for several levels of noise and several covariance priors. Interpret the results.

3 Theory

3.1 Multiple linear regression via maximum likelihood

5 Show that the ML estimates for the weights are obtained by

$$\widehat{\boldsymbol{w}} = \left(X^\top X\right)^{-1} X^\top \boldsymbol{y},$$

where X stands for the training input matrix (with training instances $\boldsymbol{x}_i$ stored row-wise), and $\boldsymbol{y}$ for the vector of associated outputs y_i.

6 We study here the distribution of the ML estimator $\widehat{\boldsymbol{w}}$ of the parameter vector $\boldsymbol{w}$.

6 a Show that for any Gaussian random vector $\boldsymbol{U} \sim \mathcal{N}(\boldsymbol{a}, B)$, then the random vector $\boldsymbol{V} = \boldsymbol{c} + D\boldsymbol{U}$ is such that $\boldsymbol{V} \sim \mathcal{N}(\boldsymbol{c} + D\boldsymbol{a}, DBD^\top)$.

6 b Show that the ML estimator $\widehat{\boldsymbol{w}}$ [1] of the parameter vector $\boldsymbol{w}$ is distributed as

$$\widehat{\boldsymbol{w}} \sim \mathcal{N}\left(\boldsymbol{w}, \sigma_n^2 \left(X^\top X\right)^{-1}\right).$$

3.2 Bayesian linear regression

We now consider a linear regression model with a Gaussian Bayesian prior on $\boldsymbol{w}$: $\boldsymbol{w} \sim \mathcal{N}(\boldsymbol{0}, \Sigma_p)$.

7 Show that the posterior distribution is

$$\boldsymbol{w} \mid X, \boldsymbol{y} \sim \mathcal{N}(\overline{\boldsymbol{w}}, A^{-1}), \quad \text{with } \overline{\boldsymbol{w}} = \sigma_n^{-2} A^{-1} X^\top \boldsymbol{y}, \quad \text{and } A = \sigma_n^{-2} X^\top X + \Sigma_p^{-1}.$$

You can use the fact that, for any symmetric (and invertible) matrix A, we have

$$\boldsymbol{z}^\top A \boldsymbol{z} + 2b^\top \boldsymbol{z} + c = (\boldsymbol{z} + A^{-1}b)^\top A (\boldsymbol{z} + A^{-1}b) - b^\top A^{-1} b + c.$$

[1] Note that the notation we use does not make a distinction between the estimator and its realization.

8 We now focus on the predictive distribution.

8 a Show that the noise-free predictive distribution is

$$\boldsymbol{f}^\star \mid X^\star, X, \boldsymbol{y} \sim \mathcal{N}(X^\star \overline{\boldsymbol{w}}, X^\star A^{-1} X^{\star\top}).$$

8 b Write the random vector containing the output variables as $\boldsymbol{Y}^\star$. Show that the predictive distribution for $\boldsymbol{Y}^\star$ is

$$\boldsymbol{Y}^\star \mid X^\star, X, \boldsymbol{y} \sim \mathcal{N}(X^\star \overline{\boldsymbol{w}}, X^\star A^{-1} X^{\star\top} + \sigma_n^2 I_m).$$

9 Show that the marginal likelihood writes as

$$p_{\boldsymbol{Y}}(\boldsymbol{y} \mid X; \sigma_n^2, \Sigma_p) = p_{\mathcal{N}}\left(\boldsymbol{y} \mid \boldsymbol{0}, \Sigma_n + X \Sigma_p X^\top\right).$$

For this purpose, you will need the Woodbury matrix identity, which writes as

$$(B + UCV)^{-1} = B^{-1} - B^{-1} U \left(C^{-1} + V B^{-1} U\right)^{-1} V B^{-1};$$

and the matrix determinant lemma, which writes as

$$\left|B + UCV^\top\right|^{-1} = \left|C^{-1} - V^\top B^{-1} U\right| |C| \, |B|.$$